From Scratch:
The Design and Implementation of a SAT Solver
(First Draft)

Oliver Lie

September 6, 2026

Contents

Preface

Hello, thank you for picking up this book. I have just finished writing it, except for this portion which I'm writing right now! It has been one epicly long journey of 9 weeks in my last quarter of college before I graduate. I wasn't even sure if I would be able to complete this by the end, but it seems just like my assignments, I finished this at the last minute.

To be completely honest, I am not an expert when it comes to computer science. After all, my degree is in environmental sciences (just kidding, I am a computer science major). However, even though I don't know everything about every data structure, algorithm, design pattern, or whatever, I think that this book would be great for someone getting started in the world of SAT solving. These things that I am not an expert in are my passion, and by doing this technical research, I hope to become a little less of "not an expert" in those fields.

We are not going to revolutionize SAT solving, in fact, we're barely going to scratch the surface in this book (it's a big world out there). What I aim to do in writing this is to help you understand how SAT solvers work and how they are implemented. I hope that this remains accessible and I'm not too confusing, however, I am just a ~~girl~~ undergraduate. Because we're covering how SAT solvers work and are implemented, if you are a student tasked with building one, don't use this to cheat. That would hurt my feelings. Instead use this to help your own understanding instead of copy-pasting.

Throughout this book we will build our own modern SAT solver from scratch, complete with all the bells and whistles. We will begin with the most basic algorithm imaginable, brute force, and gradually work our way through DPLL, CDCL, and beyond.

Throughout this journey we will encounter thought experiments, exercises, and a handful of experiments intended to solidify our understanding. Any external resources will be cited (hopefully correctly), and in keeping with the goal of accessibility:

- Questions that are explicitly asked will eventually be answered.
- Proofs will be explained in informal language before becoming formal.
- Code will be provided and linked through GitHub (or another hosting platform).
- Revisions to this book will be made whenever I discover mistakes or better explanations.

There is one more thing that deserves acknowledgement: some pieces of code were AI-assisted. To be completely transparent, "AI-assisted" means that instead of spending

days repeatedly banging my head against a wall trying to understand a bug or some subtle algorithmic detail, I occasionally asked an AI to explain concepts or help debug an implementation. Every word of this book, however, is my own. AI did not generate any of the explanatory text you will read here, although it certainly helped produce some of the code that accompanies it. I know that AI is very unpopular, and I do get that, however, even with literature and code I could find, it wasn't enough **for me** to figure out how everything is implemented from the ground up. That being said, I would still encourage you to read the other literature out there and look at code on your own, and perhaps you'll find me an idiot for needing AI assistance in the first place.

There are practical reasons for this. SAT solvers are difficult to implement correctly, and introducing subtle bugs is remarkably easy. In fact, before deciding to write this book, I had already attempted to build a SAT solver once before. I never managed to get a working CDCL implementation.

That failure, however, taught me a tremendous amount. So in a sense, we are learning SAT solving together (isn't that comforting?). My previous mistakes have shown me many of the pitfalls, and my hope is that by documenting this journey I can help you avoid making the same ones. So without further ado, let's begin. By the time we reach the end, I hope I will have taught you more than I myself learned.

This version of *From Scratch* is the second draft. In this version, we are going to improve our handling of pseudocode, and better explore differences between pseudocode and the actual implementation in our SAT solver. I hope that this helps you understand the material better.

Chapter 1

Introduction

SAT solvers, in my opinion, are one of the most underrated pieces of algorithmic technology of the modern day. They are used in both software and hardware verification (circuitry testing), product configuration, package management, computational biology, particle physics, solving graph problems, and much much more. [1]

In a more computer science-focused world, they have applications in NP Completeness. Since (nearly) [2] every problem has a reduction to a Boolean SAT problem, creating one really really good SAT solver lets us solve all those problems in a "fast" manner. So they are important everywhere despite the vast majority of us never really thinking about their existence. One note on what "fast" means: even if solving a SAT problem takes a small amount of time, the algorithmic complexity isn't necessarily "fast" as there is no known polynomial algorithm for any NP-Complete problem (I am a firm believer that $P = NP$, and yes I know that makes me crazy).

Before we start writing out code, proofs, and the like, we need to make sure we're all on the same page on things such as input and ideas. Reading through this introduction is important so that you understand the basic notation used throughout this book.

One last thing to note: we will write out pseudocode rather than the code for one specific programming language. In the first draft, I used a C-based syntax for readability purposes. However, what I thought was a readability issue was actually my lack of ability to properly format pseudocode using LaTeX. So in this version, we will use actual pseudocode syntax, and therefore, some proficiency with pseudocode is required to read this. Luckily, I will still explain what's going on, and I think you'll be able to pick it up quite easily. Just like in the first version, we will pretend our arrays and other data structures are dynamically resizable and don't require any pre-allocation of space. [3] That being said, you will see "our interface" sections every now and then when we make changes to our SAT solver object. The syntax in those sections will be more C++ based than pseudocode based because that is part of our actual implementation.

[1] `https://www.cs.utexas.edu/~isil/cs389L/lecture4-6up.pdf`

[2] Before anyone comes after me, the Halting Problem doesn't have a reduction to a boolean satisfiability problem, so take that.

[3] If you are implementing this in real code, this tiny detail of no pre-allocation of memory will come back to bite you later, so if you are encountering segfaults, this *should* be one of the first things you should check!

1.1 What Do SAT Solvers Even Solve?

"What do SAT solvers even solve," I hear you say? SAT solvers solve *boolean satisfiability* problems. Another way to say that is they solve *propositional logic* problems. There are many forms of propositional logic, and you may be familiar with it. Here are some basic examples:

- $P \rightarrow Q$, an implication statement
- $(A \wedge B)$, an and statement
- $(A \vee B)$, an or statement
- $\neg A$, a negation/not statement

Of course these are very basic examples, and we haven't included every operator such as xor, nand, etc. As you know, we can rewrite implications to contain only and, or, and not operators, which is what SAT solvers operate on. In other words, SAT solvers only operate on Boolean algebra equations. [4]

However, SAT solvers don't operate on just any Boolean algebra equation. No no no, they must be in a proper form, called CNF (more on that in the next section). CNF (Conjunctive Normal Form) equations are a "conjunction of disjunctions," i.e., clauses of ors, combined with ands.

Some examples of CNF are as follows:

- $(A \vee B)$
- $(A \vee B) \wedge (\neg C)$
- $(A \vee B) \wedge (\neg C \vee D)$

Some examples that are **not** CNF:

- $\neg(A \vee B)$, where there is a NOT in front of a clause
- $\neg(A \vee B) \wedge (C)$, where there is a NOT in front of a clause
- $(A \vee B \wedge C)$, where there is an AND within a clause
- $(A \wedge B) \vee (C)$, where it is in DNF form. [5]

[4] A Boolean algebra equation is one containing only and ($\wedge$), or ($\vee$), and not ($\neg$) operations (at a basic level).

[5] DNF (Disjunctive Normal Form) is a "disjunction of conjunctions," i.e., clauses of ands combined by ors. Solving these is trivially easy, as we just need to find one clause where no element evaluates to false.

1.2 Our Environment

Before we begin, we should talk about the environment that this book uses.

- Hardware: an Apple M1 MacBook Air, running MacOS Sequoia 15.7.8 with 8GB RAM and 512GB of storage space.
- Programming Language: C++20 compiled by Apple clang version 17.0.0 (clang-1700.6.4.2) targetted for arm64-apple-darwin24.6.0
- IDE: Visual Studio Code version 1.132.1
- Testing and Benchmarking: Testing and benchmarking is done by Test++, a C++ unit testing program I wrote (and is open source in case you want to try it out). [6] As of version 20.1.3, the program itself is compiled to be optimized, and you can choose to set the optimization flags of the program you're compiling. This potentially can change the runtime of our program. This is important because we will be using the amount of time Test++ takes to finish a test suite as the time our solver took to solve each problem, even though Test++ is not technically a benchmarking program. The way tests are set up is that each category of test/testing-set is given its own Test++ test which is run to completion. The resulting time to complete that Test++ test is used as the benchmark time. Differences in benchmarking frameworks and the way tests are set up may affect the result, but our testing workflow is as follows:

    1. Create a Test++ testing function
    2. Read in the testing file(s) from the given file path/directory
    3. Create an instance of our SAT solver
    4. Solve the equation
    5. Check that the result is correct
    6. Repeat steps 3-5 until no more testing files need to be processed

1.3 CNF Encoding

Now that we know what our inputs look like, the next question (I hope you're asking questions) is "how do we encode that?" Well good thing you're reading this, because that's what we're here to talk about! In this section, we'll expand our base of boolean algebra, including some definitions, transformations, and finally good encoding.

Definitions We'll Use

Clause A clause is a collection of variables, in which each variable is separated by an or ($\vee$), and each clause is separated by an and ($\wedge$).

[6] Not a shameless plug because it's awesome.

Variable A variable is an element of a clause, taking on a value of True, False, or Unassigned. Each variable can be negated ($\neg$) or not, affecting its evaluated value.

Literal A literal is an evaluated variable, evaluating to one of True, False, or Unassigned. Since a literal is an evaluated variable, it cannot be negated. A literal whose corresponding variable is unassigned evaluates to Unassigned, regardless of whether or not the variable is negated.

A quick example of the difference of a literal value, using all three of our definitions:

$$(\neg A)$$

is a clause of one variable "A", which is negated. If the variable $A = False$, then its literal value is $True$, because

$$\neg False \equiv True.$$

In other words, A was assigned the value False, and then it evaluated to True. This will be very important in the future, so please take the time to understand it well enough.

As discussed earlier, a CNF equation only has three operators: and ($\wedge$), or ($\vee$), and not ($\neg$), but in propositional logic, there are many more operators, and their clauses are not guaranteed to be in the format we want. There are many algorithms to transform any propositional logic equation into CNF, but frankly, that's beyond the scope of this book (it's also beyond my scope). Instead, we will just look at how we transform each operator into an equivalent format using just our three operators.

Implication ($\rightarrow$)

p	q	$\neg p$	$p \rightarrow q$	$\neg p \vee q$
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Xor ($\oplus$)

p	q	$\neg p$	$\neg q$	$p \oplus q$	$p \vee q$	$\neg p \vee \neg q$	$(p \vee q) \wedge (\neg p \vee \neg q)$
T	T	F	F	F	T	F	F
T	F	F	T	T	T	T	T
F	T	T	F	T	T	T	T
F	F	T	T	F	F	T	F

Biconditional ($\leftrightarrow$)

p	q	$p \leftrightarrow q$	$p \rightarrow q$	$q \rightarrow p$	$(\neg p \vee q) \wedge (\neg q \vee p)$
T	T	T	T	T	T
T	F	F	F	T	F
F	T	F	T	F	F
F	F	T	T	T	T

We won't look at negations of operators such as nand, nor, xnor, or any others. I'm not a boolean algebra expert when it comes to what operators exist. But if you were given an equation with these extra operators, you could easily apply the correct equivalency, then apply an algorithm that converts any boolean algebra equation into one of CNF form.[7]

DIMACS CNF Format

Now that we know what CNF looks like, we can now discuss how it's formatted for a SAT solver's consumption. Typically, CNF is formatted in *DIMACS CNF* form.

Some rules for DIMACS CNF form are:

1. The file may begin with comment lines. The first character of each comment line must be a lower case letter "c". Comment lines typically occur in one section at the beginning of the file, but are allowed to appear throughout the file.
2. The comment lines are followed by the "problem" line. This begins with a lower case "p" followed by a space, followed by the problem type, which for CNF files is "cnf", followed by the number of variables followed by the number of clauses.
3. The remainder of the file contains lines defining the clauses, one by one.
4. A clause is defined by listing the index of each positive literal, and the negative index of each negative literal. Indices are 1-based, and for obvious reasons the index 0 is not allowed.
5. The definition of a clause may extend beyond a single line of text.
6. The definition of a clause is terminated by a final value of "0".
7. The file terminates after the last clause is defined.

There are some things about this syntax to be wary of:

1. The definition of the next clause normally begins on a new line, but may follow, on the same line, the "0" that marks the end of the previous clause.
2. In some examples of CNF files, the definition of the last clause is not terminated by a final "0".
3. In some examples of CNF files, the rule that the variables are numbered from 1 to N is not followed. The file might declare that there are 10 variables, for instance, but allow them to be numbered 2 through 11.
4. Comments can appear anywhere in the file, but the comment must be on its own line and it must start with "c". [8]

For a quick example, here is a simple way to format a DIMACS CNF equation:

[7] I honestly may be talking out of my ass here. I will do more research on this subject at a later date.

[8] `https://jix.github.io/varisat/manual/0.2.0/formats/dimacs.html`

```
p cnf 4 2
1 2 -3 0
-1 4 0
```

And one with comments:

```
c this is a comment
p cnf 4 2
c this is another comment
1 2 -3 0
-1 4 0
```

In order to increase the compatibility of our SAT solver, we will be bending some rules.

We will firstly allow missing header lines. In the case where no header is passed in, we will instead infer the number of clauses and variables. We will also be very lenient in where comments can appear, but we will enforce that variables must be between 1 and N.

1. Comments can appear anywhere in the file, but they **MUST** be on their own line and start with "c".
2. A header is not necessary, and if one is not provided, then the number of variables and clauses shall be inferred. However, if one is provided, then the number of variables and clauses **MUST** match the header.
3. All clauses, with the exception of the last clause, must end with "0", and can extend multiple lines.
4. All variables are numbered 1 through N, where N is the number of variables. That is, if you say there are 10 variables, they must be labeled $1, 2, \ldots, 10$ and not $2, 3, \ldots, 11$ or any other convention.
5. (For compatibility) Upon encountering a "%" character outside of a comment, we will stop parsing immediately.

Thus, valid input can take on many forms:

```
p cnf 4 2
1 2 -3 0
-1 4 0
```

```
4 2
1 2 -3 0
-1 4 0
```

```
p cnf 4 2
c this comment doesn't have to be at the top
1 2 -3 0
-1 4 0
```

```
p cnf 4 2
c this comment doesn't have to be at the top
1 2 -3 0
-1 4
c the first two numbers are still the number
c of variables and the number of clauses
```

```
p cnf 4 2 1 2 -3 0
-1 4 0
```

More information can be found here.[9]

1.4 Reading a CNF Equation Programmatically

Now that we know what our input looks like, we can finally discuss reading it programmatically. Firstly we must discuss how our SAT solver will be represented in code. At the moment, we just need to keep track of

- the number of variables
- the number of clauses
- each clause in our equation
- what our solution is

```
1  public interface:
2      SATSolver(string input)
3      bool solveCNF()
4
5  private interface:
6      void parse(string equation)
7      int numVars
8      int numClauses
9      int[][] clauses
10     optional<bool>[] vars
```

Our pseudocode is as follows:

[9] https://people.sc.fsu.edu/~jburkardt/data/cnf/cnf.html

Algorithm 1 SAT solver constructor

```
1: procedure CONSTRUCTOR(input)
2:     if input is a file then
3:         content ← READFILE(input)
4:         PARSE(content)
5:     else                                  ▷ We assume input is a CNF equation
6:         PARSE(input)
7:     end if
8: end procedure
```

Algorithm 2 Input parsing

```
1:  procedure PARSE(equation)
2:      if equation contains "p cnf" then
3:          A[1..n]
4:          for line in input do
5:              if line starts with "c" then
6:                  continue on to the next line
7:              else
8:                  for token in line do APPEND(A, token)
9:                  end for
10:             end if
11:         end for
12:         numVars ← A[1]
13:         numClauses ← A[2]
14:         idx ← 2
15:         for i ← 1 up to numClauses do
16:             Clause[1..n]
17:             while A[i] ≠ 0 do APPEND(Clause, A[i])
18:                 idx ← idx + 1
19:             end whileAPPEND(Clauses, Clause)
20:         end for
21:     else
22:         numVars ← 0
23:         numClauses ← 0
24:         A[1..n]
25:         for line in input do
26:             if line starts with "c" then
27:                 continue on to next line
28:             else
29:                 for token in line do
30:                     numVars ← MAX(numVars, token)
31:                     if token = 0 then
32:                         numClauses ← numClauses + 1
33:                     end ifAPPEND(A, token)
34:                 end for
35:             end if
36:         end for
37:         idx ← 0
38:         for i ← 1 up to numClauses do
39:             while A[i] ≠ 0 do
40:                 APPEND(Clause, A[idx])
41:                 idx ← idx + 1
42:             end while
          APPEND(Clauses, Clause)
43:         end for
44:     end if
45: end procedure
```

This code will be implemented slightly differently in the example code, as I am using C++. There is also a big bug, which is that the code assumes that because "p cnf" appears in the file, there is a header. This is not always true because it does not consider whether or not the header is on the same line as a comment, so this will break it:

```
c p cnf
1 2 -3 0
-1 4 0
```

Because it assumes that there is a header when in fact there is not. I will leave fixing this bug as an exercise to both the reader and the writer, however, I will not be implementing this fix because if you're purposely trying to break the constructor, you're not trying to solve a CNF equation anyways. In case you're wondering, the fix is to first find "p cnf" then scan backwards until you reach either the start of the file, or the end of the previous line. If you don't encounter a "c" then you know that this header is actually a header. [10]

[10] What if there are multiple "p cnf" in the equation? They all better be commented is all I have to say on that matter. In my implementation I try to stop the user from shooting themselves in the foot if they try to shoot themselves in the foot, however, it's completely valid to just let the user shoot themselves in the foot and learn not to do that.

Chapter 2

Brute Force

Since the dawn of man, whenever encountered with a difficult problem, we have decided "eh, let's do it in the hardest way possible because I can't think of a better solution." The history of brute forcing things goes all the way back to our ancestors discovering fire, [1] all the way to our current solution to climate change. [2]

The idea of brute forcing something is really as simple as it gets. We have n number of literals, and therefore 2^n possible assignments, and we will check them all until we have either found a solution, or until we've run out of assignments to check. Because of how simple the idea is, it would just be better to show code and explain that instead of explaining then showing code.

[1] This is probably not true.

[2] Which is doing nothing and hoping everything sorts itself out, which is really disappointing.

Algorithm 3 Brute Force

```
 1: procedure SOLVEBRUTEFORCE
 2:     for each possible assignment[1..numVars] do
 3:         solved ← True
 4:         for Clause[1..n] in Clauses do
 5:             satisfied ← False
 6:             for i ← 1 up to n do
 7:                 v ← Clause[i]
 8:                 idx ← ABS(v)
 9:                 if v > 0 then
10:                     satisfied ← assignment[idx] ∨ satisfied
11:                 else
12:                     satisfied ← ¬assignment[idx] ∨ satisfied
13:                 end if
14:             end for
15:             solved ← satisfied ∧ solved
16:         end for
17:         if solved = True then
18:             vars ← assignment
19:             return True
20:         end if
21:     end for
22:     return False
23: end procedure
```

Let's understand what's going on. Firstly, an *assignment* in this case is an array of boolean values. Under the current assignment, we assume that it's satisfactory (innocent until proven guilty) as shown on **line 3**. Next, we iterate through each clause in our database, and we assume that the clause is not satisfied. This is because or-ing a false value with a true value will get us a satisfied clause, but or-ing a true value with all false values will keep us at true. And if we decided to "and" everything together, it would have its own issues as well.

On **line 6**, we iterate through the current clause. We grab its literals at index *i*, and then we take the absolute value of that literal. This is because the absolute values of our literals ranges from [1..*numVars*] and each assignment has exactly *numVars* variables in it. So by taking the absolute value of a literal, we get to use it as an index into our *assignment* variable.

On **line 9**, we check if the literal is greater than 0. If it is greater than 0, we just "or" its assignment's value with whether or not the clause is currently satisfied. Otherwise, we take the negation of the assignment's value and "or" that with whether or not the clause is currently satisfied. This is because negative literals means "use the opposite value of whatever I've been assigned with." Then we "and" that clause-satisfied value with whether or not the equation has been solved. We repeat this until there are no more clauses.

Then on **line 17**, we check if we've solved the equation. If we have, we store that

assignment in our SAT solver and return *true* to signify that we've found a solution. Otherwise, we move on to the next assignment. We do that until there are no more assignments or we've found a satisfactory assignment. If we reach a point where we've looked through every assignment and none of them are satisfactory, we return $false$ on **line 22**.

Since it's trivially easy to see why the code works, we will move onto something more interesting: optimizing brute force.

2.1 Optimizing Brute Force

I'll be completely honest with you, if you want to skip this section, go ahead. It's more or less just something interesting enough we can do to make the (arguably) worst algorithm[3] better.

Since we're dealing with pseudocode, we're not going to consider the hardware side of computers such as branch prediction, cache locality, or anything that the hardware uses to execute our code (other than the CPU). We will consider it from a pure Big-O perspective (excluding constant factors unless we're choosing between two algorithms with the same computational complexity).

The first thing we will add is short circuiting to our clause evaluation. After **line 15** where we finish OR-ing the "clause satisfied" boolean, we will add in a new block:

[3] I do not know of any algorithm worse than brute forcing.

Algorithm 4 Brute Force

```
 1: procedure SolveBruteForce
 2:     for each possible assignment[1..numVars] do
 3:         solved ← True
 4:         for Clause[1..n] in Clauses do
 5:             satisfied ← False
 6:             for i ← 1 up to n do
 7:                 v ← Clause[i]
 8:                 idx ← ABS(v)
 9:                 if v > 0 then
10:                     satisfied ← assignment[idx] ∨ satisfied
11:                 else
12:                     satisfied ← ¬assignment[idx] ∨ satisfied
13:                 end if
14:                 if satisfied = True then
15:                     Move onto the next clause
16:                 end if
17:             end for
18:             solved ← satisfied ∧ solved
19:         end for
20:         if solved = True then
21:             vars ← assignment
22:             return True
23:         end if
24:     end for
25:     return False
26: end procedure
```

This just says "once this clause is satisfied, go evaluate the next one immediately."

We can then add another conditional after this loop, which will move onto the next variable assignment the instant the equation is no longer satisfied:

Algorithm 5 Brute Force

```
 1: procedure SOLVEBRUTEFORCE
 2:     for each possible assignment[1..numVars] do
 3:         solved ← True
 4:         for Clause[1..n] in Clauses do
 5:             satisfied ← False
 6:             for i ← 1 up to n do
 7:                 v ← Clause[i]
 8:                 idx ← ABS(v)
 9:                 if v > 0 then
10:                     satisfied ← assignment[idx] ∨ satisfied
11:                 else
12:                     satisfied ← ¬assignment[idx] ∨ satisfied
13:                 end if
14:                 if satisfied = True then
15:                     Move onto the next clause
16:                 end if
17:             end for
18:             solved ← satisfied ∧ solved
19:             if solved = false then
20:                 Move onto the next assignment
21:             end if
22:         end for
23:         if solved = True then
24:             vars ← assignment
25:             return True
26:         end if
27:     end for
28:     return False
29: end procedure
```

With that, that's all the optimization I can think of. In case you're wondering, no this does not change the asymptotic runtime of the method, however, it's easy to see why these changes would presumably make the algorithm run faster as we're adding code that stops evaluation the moment we've figured just enough out to know if it's worth continuing evaluation and deferring extra calculation (recording the current assignment) until once we know it's satisfiable.

As an exercise to the reader, I will ask you, how can you parallelize this algorithm? And once you've figured that out, can you program it in a way where once one thread finds a satisfactory assignment, it terminates all other threads immediately instead of waiting for the others to finish?

2.1.1 Generating Assignments

How do we generate assignments? In the C++ implementation, we don't generate an assignment directly. Instead, we can recognize that a string of 1's (true) and 0's (false) is just a string of bits, which is just a datatype. We can use a large data, such as a *long* to hold 32 variable assignments. Since this is just a data type, if we had a for loop from $i \rightarrow 2^32$, or however many variables we have involved, our index variable can be used as an assignment itself! Using some clever bit shifting, we can either store the assignment into a bit array or index the bits of i directly to simulate the behavior of accessing the assignment of a variable. You may think that 32 variables, or 64, depending on your compiler and architecture is very small, and while that may be true, you must consider how inefficient of an algorithm brute force is to begin with. Theoretically we *could* make brute force numerically scale to any number of inputs if we wanted to, but the algorithm is so terrible that after 32, it would become basically unusable anyways other than a potential CPU hog.

2.2 Brute Force - Results

To see how well our current algorithm does, running some tests would be a good idea. Why run tests when we have already proven that our algorithm works "perfectly?" To see how quickly it runs of course.

To test its speed (and as we add more features, to ensure correctness is kept), we use a database of problems from here:

`https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html`

We gave our current implementation 1000 SAT instances (uf20-91), each with 20 variables and 91 clauses each, all satisfiable. It took this baseline brute force algorithm **2,119,301 milliseconds**, which is roughly 35.32 minutes. This result is actually from the first time I wrote a brute force algorithm. The brute force result you can find on GitHub, it actually took **1,770,851 milliseconds**, which is about 29.51 minutes. I have no idea what I did in the rewrite that shaved off 4 minutes. I won't even lie to you reader, I am a busy undergraduate student, so I didn't even do the "best practice" for testing, which would be to quit all other processes and then benchmark it. However, while I was testing, I was doing a Canvas assignment, so I was doing lots of background work.

Normally in science you have to run an experiment at least three times for data accuracy. But you must understand that I don't want to give up an hour and a half of my life to an algorithm that we're going to be improving upon anyways.

Here are the results for all brute force algorithms, including the short-circuited and parallel versions:

```
On all 1000 equations in uf20-91:

Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).
```

For the parallel version, the way it works is that it spawns in some number of worker threads and they all share a flag that alerts them if any other thread has found a solution. If a thread has, each worker will finish whatever iteration it's on and terminate. The full code can be found on the GitHub.For the parallel version, the implementation isn't perfect. I believe that we don't prevent false sharing amongst cache lines (ie we get more cache swaps between threads leading to more cache misses), which could explain why on my M1 MacBook Air, we don't get close to a 8 times speedup. Since this was for fun, I'm not going to perfect the parallelized version, that can be an exercise to the reader :)